HG2J-7U
Linux Setup and Exploration

Hardware, filesystem, SSH access, and outstanding work

Status date: 18 October 2026

This document describes the Linux system underneath the IDEC HG2J-7U touchscreen interface, the changes made to gain access to it, and what the experiments have established so far. It is a practical record of this particular HMI and project, not a claim that every HG2J firmware image has the same contents.

The main result is that SSH access works and gives access to a small but capable Linux computer. The original IDEC interface continues to run. Linux can open an additional terminal on the screen, read touchscreen events, and run compatible ARM programs. Several input and development facilities remain unavailable or unresolved.

Scope of the setup script. `setup_hmi_dropbear.py` enables SSH and configures the service account in an extracted OS archive. It is *not yet a general-purpose OS provisioning tool*. It does not install uinput modules, compiler, or solve keyboard input issues.

1 The system at a glance

Item	Recorded configuration
Device	IDEC HG2J-7U; device tree identifies a TI AM335x IDEC 7-inch wide PD.
Processor	One 32-bit ARMv7 core; recorded frequency 600 MHz. ARM hard-float 32 bit programs.
Operating system	Arago 2019.07, an embedded Linux distribution.
Kernel	Linux 4.19.59 with IDEC-specific build; built 29 November 2024.
Memory	248,564 KiB visible to Linux (about 243 MiB); no swap configured.
Root storage	About 129.7 MiB usable root filesystem; about 74.7 MiB available in the supplied snapshot.
Display	800 by 480 pixels; LVDS-1 output; 16-bit framebuffer reported.
Graphics	Weston 5.0.0 with Wayland and the PowerVR graphics stack.
Shell and tools	`/bin/sh`; many commands supplied by BusyBox 1.29.3.
Example network	HMI `192.168.1.20`; directly connected PC `192.168.1.160/24`.

2 Understanding the filesystem

Linux presents storage and hardware through one directory tree. A path beneath `/` is not necessarily stored on the same device as the root filesystem. Some directories are separate flash partitions, others are USB storage, and several exist only in RAM.

Path	Size	Meaning in the recorded system
`/`	129.7 MiB	UBIFS on `ubi0:rootfs-a`; persistent OS files, including `/etc` and normally `/home/service`.
`/home/idec/rom`	39.7 MiB	YAFFS2 flash partition, `/dev/mtdblock16`; mounted with `noexec`.
`/mnt/work`	2.0 MiB	YAFFS2 flash partition, `/dev/mtdblock17`; also `noexec`.
`/run`	121.4 MiB	RAM-backed runtime data.
`/var/volatile`	50 MiB	RAM-backed temporary data and logs.
`/home/idec/.tmp`	20 MiB	RAM-backed area containing the running IDEC application in the observations.
`/home/idec/.work`	10 MiB	RAM-backed working area.
`/home/idec/usr/tmp`	100 MiB	RAM-backed application workspace; `noexec`.
`/home/idec/usr/exmem/A`	USB size	USB-A storage; observed as `/dev/sda1`, FAT, about 14.8 GiB, with `noexec`.

2.1 What survives a restart?

Files in a `tmpfs` filesystem live in RAM and disappear when the system restarts. Its reported size is a limit, not a separate pool of extra memory: these files compete with programs for the same RAM. Save useful logs to the PC or mounted USB drive before restarting.

USB-Media files normally survive a restart, but an OS or project download can replace them. The partition list includes A and B kernel/rootfs names.

2.2 Useful read-only checks on the HMI

```
df -h
cat /proc/mounts
cat /proc/meminfo
cat /proc/mtd
cat /proc/filesystems
ls -l /home/idec/usr/exmem/A
```

`/proc` reports live kernel and process information. `/sys` exposes device settings and status. `/dev` contains device access points. These are interfaces to the running system, not files to edit.

3 What the setup changes

Dropbear, a compact SSH server, was already present in the extracted OS files. This project enables and configures it rather than installing a replacement Linux system. SSH provides a remote command prompt over the network. A brief effort ro determine how WindO/I-NV4 may enable the existing ssh installation. No evidence was found. However, in `/etc/profile` evidence for yocto and file transfer via an open-palm eclipse plug-in were present. The exact configuration to enable that mode of access was not explored.

File or setting	Behavior of the inspected setup scripts
`/etc/default/dropbear`	Writes `NO_START=0`, `DROPBEAR_PORT=22`, and empty `DROPBEAR_EXTRA_ARGS`. Password login is enabled by removing the earlier `-s` restriction. The earlier `-w` root-login restriction is also absent.
`/etc/passwd`	Adds or replaces `service` with UID 0, GID 0, home `/home/service`, and shell `/bin/sh`.
`/etc/group`	Adds or replaces the named `service` group with GID 1001.
`/etc/shadow`	Stores the supplied SHA-512 password hash for `service`; archive mode is set to 0600.
`/home/service`	Creates or updates the directory with mode 0700 and archive ownership 1001:1001.
`/etc/rc5.d/S20dropbear`	Adds or updates a startup link to `../init.d/dropbear`.
Optional host key	`--host-key` installs an existing RSA Dropbear host key at `/etc/dropbear/dropbear_rsa_host_key`, owned by root with mode 0600. The script does not generate it.
`/etc/profile`	Only the later `setup_hmi_dropbear_path.py` variant changes PATH, adding `/sbin` and `/usr/sbin`.

3.1 The account name does not determine its privileges

The active account line in both inspected variants is:

```
service:x:0:0:Service SSH:/home/service:/bin/sh
```

UID 0 means full root authority. Logging in as `service` therefore gives administrative access even if the environment still says `USER=service`. The earlier ordinary-user design used UID/GID 1001, but that is not the active account line in these versions. Use `id` to check actual privileges.

3.2 How the archive is updated

The script requires an extracted `os_update` directory and its adjacent `os_update.tar.xz`. It reads the existing archive, preserves member metadata except for deliberate changes, checks selected entries, then replaces the archive and updates corresponding extracted files.

Edits in the extracted directory are not automatically packed. The archive is the source of truth for unchanged members. `setup_hmi_dropbear.py` does not setup `authorized_keys`, install uinput, change the kernel, or supply a package manager at this time.

4 Connecting by SSH

Commands in this section are labeled by where they run. The PC prepares the network connection; the HMI commands run after logging in.

4.1 PC: direct Ethernet connection

The recorded example uses `enp2s0` on the PC. Replace this with the actual Ethernet interface if different. The added address is temporary.

```
sudo ip addr add 192.168.1.160/24 dev enp2s0
sudo ip link set enp2s0 up
ssh -o IdentitiesOnly=yes -o HostKeyAlgorithms=+ssh-rsa \
    service@192.168.1.20
```

The HMI offered an older RSA host-key algorithm, so the client needed the `HostKeyAlgorithms` exception shown above. Password authentication succeeded in the logs. Earlier public-key attempts reported an unknown algorithm, and installations using public keys resulted in the HMI booting into recovery mode.

Some earlier experiments used `192.168.1.150`. A downloaded project was observed to restore `192.168.1.20`. Check the device's current address rather than assuming the address before and after a download will be identical. These settings are configurable in the WindO/I-NV4 application interface.

4.2 HMI: verify the session

```
id
cat /etc/os-release
uname -r
printf '%s\n' "$PATH"
netstat -ltn
```

The supplied environment has the original short PATH, `/usr/local/bin:/usr/bin:/bin`. PATH is the list of directories the shell searches when a command is typed without its full filename. Missing administrative directories can make an installed command appear unavailable. For the current session:

```
export PATH="/sbin:/usr/sbin:/usr/local/bin:/usr/bin:/bin"
```

4.3 Network services observed

TCP port	Purpose	Recorded behavior
22	Dropbear SSH	Listening after the setup changes.
80	HMI web service	Listening in the captured system.
2537	IDEC maintenance	Listening; used by project maintenance tools.
2539	Update FTP control	Opened temporarily as part of the download procedure.

Later project testing confirmed that a Linux program on the HMI could access the maintenance service through the HMI's own Ethernet address. Where the localhost `127.0.0.1:2537` connection was rejected. This is an IDEC application `idec_operation.BIN` capability, not an extra feature installed by the SSH script.

5 Software, graphics, and ordinary file exploration

5.1 A small runtime system, not a development workstation

The installed libraries support running software, but searches did not find Python, a C/C++ compiler, or the usual development headers. The GCC version in `/proc/version` describes the compiler used to build the kernel, while no compilers were found.

Component	Evidence and practical meaning
BusyBox 1.29.3	Provides many familiar commands with fewer options than desktop Linux tools.
C/C++ runtime	glibc 2.28 loader `/lib/ld-linux-armhf.so.3`; `libstdc++.so.6` present. Programs must match the architecture and library requirements.
Wayland / Weston	Weston 5.0.0, `weston-terminal`, `weston-info`, `weston-calibrator`, and runtime libraries are present.
Graphics libraries	EGL, OpenGL ES, GBM, DRM, PowerVR, Cairo, PNG and JPEG libraries are present.
Other libraries	ALSA and networking libraries exist.
Development/debugging	No native compiler or Python found; only a few gdb file artifacts were found.

5.2 Open a terminal on the HMI screen

Weston is the program that places application windows on the screen. It is already running alongside the IDEC application. From the SSH shell:

```
XDG_RUNTIME_DIR=/tmp/0-runtime-dir \
WAYLAND_DISPLAY=wayland-0 \
weston-terminal >/tmp/weston-terminal.log 2>&1 &
```

These settings tell the terminal which existing graphical session to use. The terminal appeared over the HMI interface during testing, sometimes after a delay. The supplied `start_waston.sh` performs this launch; despite its name, it does not start a new Weston server. It has a Bash shebang, although the command itself can be run with `/bin/sh`.

This Weston terminal accepts `--shell=NAME`, not the tested `-e` option. Launching a program window and successfully controlling it are separate results. The Vitetris experiment demonstrated the former; input remained troublesome.

5.3 Logs and backlight

```
cat /var/volatile/log/messages
tail -80 /var/volatile/log/weston.log
cat /sys/class/backlight/backlight/brightness
cat /sys/class/backlight/backlight/max_brightness
# Example value recorded in brightness.txt:
echo 9000 > /sys/class/backlight/backlight/brightness
```

The inventory reported maximum brightness 65535 and a sampled setting of 1024. A brightness write is a live adjustment, not a documented persistent setup change. The IDEC application may also control the backlight.

Missing cursor images and Compose-table warnings occurred while the terminal still opened. Likewise, Weston failed to find some GBM backends but then loaded `gbm_pvr.so`. Individual warning lines should be read with the subsequent results, rather than treated as proof that graphics are absent.

6 Touchscreen, keyboard, and uinput

6.1 Touchscreen reading

The built-in touchscreen is identified as `hy461x_ts`, connected through I2C bus 1 at address `0x38`. It appeared as `/dev/input/event1`. The beeper appeared as `event0`; its `kbd` handler does not make it a physical keyboard. Device numbers should be checked again when hardware changes.

```
cat /proc/bus/input/devices
hexdump -v -C /dev/input/event1
```

Touching the screen produced binary event records. In the captured 32-bit format, each record was 16 bytes: timestamp, event type, event code, and value. Type 3 reports an absolute position. Codes `0x35` and `0x36` report multitouch X and Y. One captured pair decoded to X=798, Y=410.

The first AWK decoder in `touchscreen.txt` used the wrong byte columns from `hexdump-C`; outputs such as 216064 are not evidence that the touch sensor has an enormous coordinate range. Raw coordinates can also be transformed by calibration before reaching the display. The log shows a `WL_CALIBRATION` setting associated with LVDS-1.

6.2 Keyboard detection did not establish usable typing

The Keychron K10 Max appeared as several USB input. Inspected udev rules identified keyboard interfaces, and Weston opened their event devices and associated them with the display. Attempts to obtain useful typing or event output were unsuccessful.

6.3 What uinput would provide

uinput lets a program create a virtual input device, such as a keyboard, and send events into Linux's input system. It would be one possible way to turn touchscreen regions into keystrokes for another application.

The recorded checks found neither a uinput devices nor a registered uinput kernel module entry:

```
ls -l /dev/uinput /dev/input/uinput 2>/dev/null
grep uinput /proc/misc
zcat /proc/config.gz | grep CONFIG_INPUT_UINPUT
# CONFIG_INPUT_UINPUT is not set
```

Confirmed limitation: the running kernel was configured without uinput. Creating a device filename alone does not supply the missing driver. **Unresolved work:** whether a compatible separately built module can be loaded on this exact kernel is untested. No successful uinput module build or load is established yet.

6.4 Other attempted input paths

A touch-processing script printed "ENTER" when a touch was detected. Forwarding that action through a terminal did not establish working game input. The on-screen keyboard was also explored, while `/etc/weston.ini` showed an empty input-method path.

7 Outstanding work and limits of the evidence

7.1 Kernel module feasibility

A kernel module is code loaded into the running kernel. It must match the actual kernel build closely; being labelled "ARM" or "Linux 4.19" is not enough. The October 17 project output recorded:

```
4.19.59-idec-hg2j-001.001.000.000-g5f8c1c6121
CONFIG_LOCALVERSION=""
CONFIG_LOCALVERSION_AUTO=y
# CONFIG_MODVERSIONS is not set
# CONFIG_MODULE_SIG is not set
# CONFIG_INPUT_UINPUT is not set
```

The expected `/lib/modules/$(uname-r)/build` path was absent. The kernel was built using GCC 8.3.0 (Arm toolchain 8.3-2019.03). These observations explain why building on the HMI is not currently practical.

The absence of module-version checks and signature support does not prove that an arbitrary module will load. Before treating uinput as deployable, obtain the matching source/configuration and build information, verify module-loading support, build for this ARM target, and demonstrate successful loading and event delivery. The inventory includes `insmod` and `modprobe`, but those command names alone do not establish kernel compatibility.

Read-only checks for the next investigation:

```
zcat /proc/config.gz | grep -E \
  'CONFIG_(MODULES|INPUT_UINPUT|MODVERSIONS|MODULE_SIG|LOCALVERSION)'
cat /proc/modules
ls -ld /lib/modules /lib/modules/"$(uname -r)"
ls -l /lib/modules/"$(uname -r)"/build
```

7.2 Open items worth carrying forward

Item	Current status / evidence needed
uinput	Not configured; no working module shown. Keep separate from SSH provisioning.
Physical keyboard	Device recognition confirmed; reliable end-to-end typing not established. Capture actual key events before assigning a cause.
On-screen input	Touch events readable; virtual keyboard and touch-to-key delivery remain experiments.
Setup script cleanup	Reconcile UID/GID, home ownership, and printed status messages. Confirm the PATH variant after deployment.
General provisioning	Future goal. Arbitrary tree edits, dependency installation, startup of custom apps, and kernel changes are not handled by the current script.
Diagnostics	`dmesg` returned "Function not implemented"; `/proc/kmsg` was absent. Preserve available application logs instead.
Performance	One sample showed `idec_operation.BIN` using about 91% CPU. Its 445 MiB VSZ is virtual address space, not 445 MiB of physical RAM.
Application lifecycle	Killing the IDEC runtime was followed by SSH disconnection. A watchdog/restart was suspected, but the cause was not proved. It is not a routine launch procedure.

8 ZNX preparation: a short command reference

These commands run on the **Linux PC**, not on the HMI. They summarize the established workflow without the history of container analysis. Use a known working ZNX for the correct device. Keep the original unchanged.

8.1 Extract the container and Linux archive

```
python3 extract_znx.py Read_regs.ZNX -o Mod_Read_regs
mkdir -p Mod_Read_regs/os_update
tar -xJf Mod_Read_regs/os_update.tar.xz \
    -C Mod_Read_regs/os_update
```

The extractor writes top-level members such as `os_update.tar.xz` and `project.znv`. It does not unpack the inner OS archive itself. The `project.znv` member contains the HMI project and can remain unchanged for an SSH-only modification.

8.2 Apply the SSH setup

Create the password hash interactively, then run the setup script:

```
umask 077
openssl passwd -6 > service.hash
python3 setup_hmi_dropbear.py Mod_Read_regs/os_update \
    --hash-file service.hash
```

Use `setup_hmi_dropbear_path.py` instead if the PATH update is desired. The inspected code requires Python 3.9 or later for `str.removeprefix`. An optional `--host-key/path/to/dropbear_rsa_host_key` supplies an existing Dropbear-format host key. This script rebuilds `os_update.tar.xz`; a separate generic tar command is not needed for its own changes.

For additional manual OS edits: the current script does not import arbitrary changes from the extracted tree. Such changes require a separate archive rebuild that preserves numeric ownership, modes and links. The old blanket `--owner=0--group=0` recipe changes all ownership and is not a general substitute for that process. An ordinary-user extraction is also not a reliable ownership-preserving source for a later full repack.

8.3 Repack and interpret the comparison

```
python3 repak_znx_roundtrip.py Read_regs.ZNX \
    Mod_Read_regs rr_ssh.ZNX
```

The repacker reads the updated top-level members, recalculates their sizes, offsets and checksums, and compares the result with the original. With unchanged members, a byte-for-byte match is the round-trip check. With deliberate OS changes, `ROUNDTRIP:DIFFERENT` is expected.

Exit-code detail: the inspected repacker returns 1 for a different output and also for errors. Read its output and confirm the intended artifact; do not treat every return code 1 as success or chain the download with `&&` expecting a modified file to return zero.

8.4 Download to the HMI

```
python3 hmi_znx_download.py rr_ssh.ZNX --host 192.168.1.20
```

Use the HMI's address *before* the update. The inspected downloader defaults to `192.168.1.150`, so an explicit host avoids ambiguity. It requests confirmation, enters System Transmission, uploads the ZNX and MD5 through the temporary FTP service, installs, and requests reboot. This replaces device software and interrupts operation. Afterwards, reconnect at the resulting IP and verify SSH, `id`, PATH, and normal HMI operation.

9 Quick reference and source notes

9.1 Useful commands on the HMI

```
# OS, kernel, and actual privilege level
cat /etc/os-release
uname -a
id

# Storage, memory, running applications
df -h
cat /proc/mounts
cat /proc/meminfo
ps -w
top

# Locate an installed administrative tool
command -v insmod
ls -l /sbin/insmod

# Input, listening services, and recent graphics messages
cat /proc/bus/input/devices
netstat -ltn
tail -80 /var/volatile/log/weston.log

hexdump -v -e '16/1 "%02x " "\n"' /dev/input/event1 |
awk '
function hex(s,    i,n) {
        n=0
        for (i=1; i<=length(s); i++)
        n=n*16 + index("0123456789abcdef", substr(s,i,1))-1
        return n
}
$9=="03" && $10=="00" && $12=="00" {
        value=hex($16 $15 $14 $13)
        if ($11=="35") x=value
        if ($11=="36") y=value
        if ($11=="35" || $11=="36") changed=1
}
$9=="00" && $10=="00" && $11=="00" && $12=="00" {
        if (changed) {
                printf "X: %d  Y: %d\n", x, y
                fflush()
                changed=0
        }
}'
```

BusyBox differences observed in the notes include unsupported `df-i`, unsupported `od-t`, and unsuitable desktop-style `psaux` usage. Brace expansion such as `event\{2,6\}` also failed in the HMI shell; use an explicit loop such as `fornin26;do...;done`.
While the Kernel is from 2019, many of the core utilities, and supporting applications are significantly older.

USB device paths:

```
sh /home/idec/usr/exmem/A/
sh /home/idec/usr/exmem/B/
```